

JD

Associate Tech Specialist / Tech Specialist - Fullstack with AI(Harb735)

Experience Range

5 - 10 Years

Position: Associate Tech Specialist / Tech Specialist- Fullstack with AI

Experience- 5 to 8 Years

Location- Remote

ROLE OVERVIEW

Client builds products that combine a modern component-based front end, a large and mature server-side JavaScript backend, and a growing suite of AI agent services. We're looking for an Associate Tech Specialist / Tech Specialist with 5+ years of experience who can move comfortably across this stack: building responsive, component-driven UIs, extending backend services and data models, and contributing to LangGraph-based agent workflows that call Claude models through AWS Bedrock.

This role suits specialist who thinks in components and state, is equally at home in older prototype-based JavaScript as in modern TypeScript, and has genuine hands-on experience building agentic AI features — not just calling a model API. You'll also be expected to use AI coding agents (like Claude Code) as part of your own workflow, while still owning, reviewing, and defending every line you ship.

KEY RESPONSIBILITIES

Design, build, and maintain responsive front-end components using Lit (or equivalent modern component frameworks), including work that crosses Shadow DOM boundaries.

Develop and extend backend services written in server-side ES5 JavaScript, alongside newer TypeScript and Python services.

Build and consume REST APIs, including authentication and role-based access control (Basic Auth, OAuth, JWT).

Model and query relational data, with attention to schema design, joins, and query performance.

Contribute to AI agent services built on LangGraph, including graph design, state management, and checkpointing.

Extend and debug Python/FastAPI services with typed, async code and Pydantic models.

Write and maintain end-to-end and API tests with Playwright (or Cypress/Selenium), including per-role permission checks.

Participate in code review, Git branching/rebasing at team scale, and diagnosing GitHub Actions workflow failures.

Bring up, debug, and operate local environments using Docker Compose.

Use AI coding agents (e.g., Claude Code) responsibly — scoping tasks clearly, reviewing generated output critically, and owning the final result.

REQUIRED SKILLS & EXPERIENCE

FRONTEND

JavaScript (ES6+) and TypeScript — strong in both.

HTML and CSS/SCSS, including responsive layout and how styling crosses a Shadow DOM boundary.

A modern component framework — Lit ideally, or solid React experience; what matters most is thinking in components, props, and state.

Web components — custom elements, Shadow DOM, reactive properties (direct experience is a plus, not a gate).

Browser fundamentals — ES modules, fetch, DevTools, and basic CSP debugging.

BACKEND

Server-side JavaScript in ES5 — prototype-based classes, no async/await; this is the largest body of code in the codebase.

REST APIs — building and consuming them, with authentication.

Relational data modelling — tables, references, joins, and query performance.

Access control — roles and permissions, Basic Auth / OAuth / JWT; a first-class concern in these applications.

AI AGENT SERVICES

LangGraph — our agent services are built on it: graphs, state, and checkpointing. LangChain or a comparable orchestration library transfers; what matters is having built something agentic, not just called a model.

Python with FastAPI — typed, async, Pydantic models; enough to extend and debug a service, not necessarily own the whole stack.

LLM application basics — prompts, tool calling, structured output, and handling cases where the model returns something unusable.

AWS Bedrock — we call Claude models through it; Anthropic or OpenAI API experience transfers.

TESTING & DELIVERY

Playwright (or Cypress / Selenium) — writing both API and end-to-end tests, including per-role permission checks.

Git at team scale — branches, rebasing, PR review, and resolving real conflicts.

GitHub Actions — enough to read a workflow and diagnose a failure.

Docker Compose — bring up and debug a local stack (up, logs, exec); no Dockerfile authoring needed.

WORKING WITH CLAUDE

Has used an AI coding agent on real work — Claude Code or similar.

Reviews what it writes — can describe a time it was confidently wrong and how they caught it.

Scopes and contextualizes tasks rather than asking for whole features in one go.

Owns the output — can explain and defend every line they submit.

NICE TO HAVE

AngularJS 1.x — our Lit components mount inside AngularJS portal widgets; optional, but rare and hard to teach, so a real bonus. (Not the same as modern Angular.)

ServiceNow — scoped apps, Service Portal, Glide APIs, Fluent SDK.

Python for deployment and provisioning scripts.

Postgres, Redis, OpenSearch; RAG or agent-evaluation experience.

Top 50 Relevant QAs.

1. Tell me about yourself and how your experience fits this role.

Answer:

I'm a full-stack engineer with experience across frontend, backend, APIs, databases, and increasingly AI-driven applications. On the frontend, I'm comfortable with component-based architectures using JavaScript and TypeScript, including state management, reusable components, responsive design, and browser APIs.

On the backend, I've worked with REST services, authentication, relational databases, and both modern and legacy JavaScript patterns. I'm also comfortable with Python services using FastAPI and Pydantic.

On the AI side, I understand that agentic systems go beyond simply calling an LLM API. I've worked with concepts such as stateful workflows, tool calling, structured outputs, retries, validation, graph-based orchestration, and checkpointing. I'm also comfortable integrating LLMs through providers such as AWS Bedrock, Anthropic, or OpenAI.

What particularly matches this position is the need to move between older JavaScript, modern TypeScript, frontend components, backend services, and AI agent workflows rather than specializing in only one layer.

JavaScript / TypeScript / Frontend

2. What is the difference between var, let, and const?

Answer:

var is function-scoped, while let and const are block-scoped.

var declarations are hoisted and initialized with undefined. let and const are also hoisted, but they remain in the **Temporal Dead Zone** until execution reaches their declaration.

const prevents reassignment of the variable binding, but it does not make objects immutable.

```
const user = { name: "John" };
```

```
user.name = "Mike"; // valid
```

```
// user = {}; // invalid
```

For modern JavaScript, I generally use `const` by default and `let` only where reassignment is necessary.

3. Explain closures in JavaScript.

Answer:

A closure occurs when an inner function retains access to variables from its outer lexical scope even after the outer function has completed execution.

```
function counter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}
```

```
const increment = counter();
```

```
console.log(increment()); // 1
```

```
console.log(increment()); // 2
```

Closures are commonly used for encapsulation, callbacks, factories, event handlers, and maintaining private state.

One thing to watch for is unintentionally retaining large objects through closures, which can contribute to memory leaks.

4. Explain the JavaScript event loop.

Answer:

JavaScript is single-threaded from an application execution perspective, but asynchronous work is delegated to browser APIs or the Node.js runtime.

The main components are:

- Call stack
- Web APIs or Node APIs

- Task/macrotask queue
- Microtask queue
- Event loop

Promises queue callbacks in the microtask queue, while operations such as `setTimeout` generally use the task/macrotask queue.

For example:

```
console.log("A");
```

```
setTimeout(() => console.log("B"), 0);
```

```
Promise.resolve().then(() => console.log("C"));
```

```
console.log("D");
```

The output is:

A

D

C

B

because promise microtasks execute before the next macrotask.

5. What is the difference between `==` and `===`?

Answer:

`==` performs type coercion before comparison, while `===` checks both type and value.

```
0 == false // true
```

```
0 === false // false
```

I prefer `===` because it avoids unexpected coercion and produces more predictable behavior.

6. Explain prototypes and prototype inheritance in JavaScript.

Answer:

JavaScript uses prototype-based inheritance.

Objects can inherit properties and methods from another object through their internal prototype chain.

In ES5:

```
function User(name) {  
  this.name = name;  
}
```

```
User.prototype.sayHello = function () {  
  return "Hello " + this.name;  
};
```

```
var user = new User("Puneet");
```

When `user.sayHello()` is called, JavaScript first checks the user object. If the method is not there, it searches `User.prototype`.

This is particularly relevant to this role because the JD explicitly says that a large part of the backend uses older prototype-based ES5 JavaScript.

7. How would you implement inheritance in ES5 JavaScript?

Answer:

```
function Animal(name) {  
  this.name = name;  
}
```

```
Animal.prototype.speak = function () {  
  return this.name + " makes a sound";  
};
```

```
function Dog(name) {  
  Animal.call(this, name);  
}
```

```
Dog.prototype = Object.create(Animal.prototype);
```

```
Dog.prototype.constructor = Dog;
```

```
Dog.prototype.bark = function () {  
  return this.name + " barks";  
};
```

Animal.call(this, name) initializes parent properties, while:

```
Dog.prototype = Object.create(Animal.prototype);
```

creates the prototype inheritance relationship.

This is essentially what modern class extends syntax abstracts.

8. How do you handle asynchronous operations in ES5 without async/await?

Answer:

Depending on what the runtime supports, I would use callbacks or Promises.

A callback example:

```
getUser(id, function (err, user) {  
  if (err) {  
    return handleError(err);  
  }  
  
  getOrders(user.id, function (err, orders) {  
    if (err) {  
      return handleError(err);  
    }  
  
    processOrders(orders);  
  });  
});
```

In a legacy codebase, I would follow the existing conventions rather than aggressively rewriting everything to modern syntax.

If Promises are available:

```
getUser(id)  
  .then(function (user) {  
    return getOrders(user.id);  
  })
```

```
.then(processOrders)
.catch(handleError);
```

9. What benefits does TypeScript provide?

Answer:

TypeScript gives JavaScript static type checking.

Major benefits include:

- Catching errors before runtime
- Better IDE autocomplete
- Safer refactoring
- Self-documenting interfaces
- Stronger API contracts
- Easier maintenance in large applications

For example:

```
interface User {
  id: number;
  name: string;
  role: "admin" | "user";
}

function getUsername(user: User): string {
  return user.name;
}
```

I still treat TypeScript as a tool rather than a replacement for runtime validation. External input still needs to be validated.

10. What is the difference between an interface and a type in TypeScript?

Answer:

They overlap significantly.

An interface is particularly good for describing object shapes and supports declaration merging.

```
interface User {
  id: number;
```

```
name: string;
}
```

A type alias can represent more than object shapes:

```
type Status = "pending" | "approved" | "rejected";
```

It can also represent unions, intersections, tuples, and mapped types.

I typically use interfaces for extensible object contracts and types for unions or more complex type composition.

Lit / Web Components / Shadow DOM

11. What is Lit?

Answer:

Lit is a lightweight library for building standards-based Web Components.

A basic component looks like:

```
import { LitElement, html, css } from "lit";
```

```
class UserCard extends LitElement {
```

```
  static properties = {
    name: { type: String }
  };

```

```
  render() {
    return html`
      <div>Hello ${this.name}</div>
    `;
  }
}
```

```
customElements.define("user-card", UserCard);
```

Lit mainly provides reactive properties, declarative templates, lifecycle hooks, and efficient DOM updates while staying close to native browser Web Component standards.

12. What are Web Components?

Answer:

Web Components are browser standards that allow reusable encapsulated UI components.

The main technologies are:

1. Custom Elements
2. Shadow DOM
3. HTML templates
4. ES modules

Example:

```
class HelloElement extends HTMLElement {  
  connectedCallback() {  
    this.innerHTML = "<p>Hello</p>";  
  }  
}
```

```
customElements.define("hello-element", HelloElement);
```

An advantage is framework independence. A Web Component can potentially be used inside React, Angular, AngularJS, or plain HTML.

13. What is Shadow DOM?

Answer:

Shadow DOM provides DOM and style encapsulation for Web Components.

For example:

```
const shadow = element.attachShadow({ mode: "open" });
```

CSS inside the shadow root generally does not leak out, and outside styles generally do not leak in.

This is useful for reusable components, but it introduces challenges for global styling, testing, selectors, and integration with older frameworks.

14. How can you style something across a Shadow DOM boundary?

Answer:

Normal selectors cannot simply reach through a Shadow DOM boundary.

Approaches include:

- CSS custom properties
- `::part()`

- `::slotted()`
- exposing configurable component properties
- placing styles inside the component's shadow root

Example:

```
my-button {
  --button-background: blue;
}
```

Inside the component:

```
button {
  background: var(--button-background);
}
```

For exposed parts:

```
<button part="button">Save</button>
```

Parent:

```
my-component::part(button) {
  font-weight: bold;
}
```

15. How does Lit know when to re-render?

Answer:

Lit tracks reactive properties.

When a reactive property changes, Lit schedules an asynchronous update and re-runs `render()`.

```
static properties = {
  count: { type: Number }
};
```

Then:

```
this.count++;
```

causes an update.

A common issue occurs when mutating an object or array without changing its reference:

```
this.items.push(newItem);
```

Lit might not recognize the desired change as expected.

Instead:

```
this.items = [...this.items, newItem];
```

creates a new reference and clearly triggers reactivity.

16. What is the difference between props and state?

Answer:

Props are generally external inputs passed into a component. State represents internal data managed by that component.

For example, a user component may receive:

userId

as a property, but internally maintain:

isLoading

error

userDetails

as state.

A useful rule is that parent components usually own shared state, while child components receive the minimum data necessary.

Backend / APIs

17. What principles do you follow when designing a REST API?

Answer:

I focus on predictable resources, HTTP semantics, consistent error handling, authentication, authorization, validation, and versionability.

For example:

GET /users

GET /users/123

POST /users

PATCH /users/123

DELETE /users/123

I use appropriate status codes:

200 OK

201 Created

204 No Content

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

500 Internal Server Error

I also avoid exposing internal stack traces or sensitive implementation details in API responses.

18. What is the difference between PUT and PATCH?

Answer:

PUT generally represents replacement of an entire resource.

PATCH represents partial modification.

For example:

PATCH /users/123

```
{  
  "name": "Puneet"  
}
```

changes just the specified field.

The exact semantics should be consistent across the API because clients depend on predictable behavior.

19. What is idempotency?

Answer:

An operation is idempotent if repeating it produces the same final state as executing it once.

For example:

PUT /users/10

with the same payload multiple times should lead to the same resource state.

GET, PUT, and DELETE are generally expected to be idempotent.

POST usually isn't.

For critical APIs such as payments, idempotency keys can also prevent duplicate processing.

Authentication / Authorization

20. Explain Basic Auth, OAuth, and JWT.

Answer:

Basic Authentication sends a username and password, normally Base64 encoded, with every request. It must always be protected with HTTPS.

OAuth 2.0 is an authorization framework allowing applications to access resources without users sharing their actual credentials with the application.

JWT is a token format. It contains claims and is cryptographically signed.

A JWT commonly contains:

```
{  
  "sub": "123",  
  "role": "admin",  
  "exp": 1780000000  
}
```

OAuth can use JWTs, but OAuth and JWT are not the same thing.

21. What is the difference between authentication and authorization?

Answer:

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

For example, a valid JWT may authenticate a user. But authorization logic must still check whether that user has permission to delete another user's account.

That distinction is especially important because this JD emphasizes per-role permissions.

22. How would you implement role-based access control?

Answer:

I would generally avoid spreading checks like:

```
if (user.role === "admin")
```

throughout the application.

Instead, I would centralize access-control rules.

For example:

admin:

users.read
users.write
users.delete

manager:

users.read
users.write

viewer:

users.read

Then an API endpoint checks a specific permission.

```
requirePermission("users.delete");
```

This scales better when roles and permissions evolve.

23. Why is hiding a frontend button not sufficient security?

Answer:

Because users can directly call the backend API.

Even if the Delete button is hidden for a normal user, they could manually send:

```
DELETE /users/123
```

with DevTools, Postman, or another script.

Authorization must therefore always be enforced server-side.

Frontend restrictions are only a usability layer.

Relational Databases

24. Explain primary keys and foreign keys.

Answer:

A primary key uniquely identifies a row.

Example:

users

id PK

name

email

A foreign key references another table.

orders

id PK

user_id FK -> users.id

amount

Foreign keys maintain referential integrity and make relationships explicit.

25. Explain INNER JOIN and LEFT JOIN.

Answer:

Given users and orders:

```
SELECT *
```

```
FROM users u
```

```
INNER JOIN orders o
```

```
ON u.id = o.user_id;
```

An INNER JOIN returns only users that have matching orders.

A LEFT JOIN:

```
SELECT *
```

```
FROM users u
```

```
LEFT JOIN orders o
```

```
ON u.id = o.user_id;
```

returns every user, including users without orders.

26. How do you investigate a slow SQL query?

Answer:

I first measure rather than guess.

I look at:

- Execution plan
- Table size
- Index usage

- Full table scans
- Join strategy
- Filters
- Sorts
- Number of rows returned
- N+1 queries
- Unnecessary columns

For PostgreSQL, I might use:

EXPLAIN ANALYZE

SELECT ...

Then I would determine whether an index, query rewrite, schema adjustment, caching strategy, or pagination improvement is appropriate.

27. What is an index, and when can an index hurt performance?

Answer:

An index is a data structure that speeds up reads by allowing the database to locate rows without scanning the entire table.

For example:

```
CREATE INDEX idx_users_email
```

```
ON users(email);
```

But indexes consume storage and make INSERT, UPDATE, and DELETE operations more expensive because the index must also be updated.

So I wouldn't blindly index every column.

28. What is the N+1 query problem?

Answer:

It occurs when the application executes one query to retrieve a collection and then executes another query for every item.

Example:

1 query to retrieve 100 users

100 additional queries for their orders

Total:

101 queries

Often this can be reduced using joins, batching, eager loading, or a DataLoader-style mechanism.

Python / FastAPI

29. Why would you choose FastAPI?

Answer:

FastAPI is particularly useful for API and AI services because it provides:

- Async support
- Python type hints
- Pydantic validation
- Automatic OpenAPI documentation
- Dependency injection
- Good performance
- Clean development experience

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class User(BaseModel):
```

```
    name: str
```

```
    age: int
```

```
@app.post("/users")
```

```
async def create_user(user: User):
```

```
    return user
```

Invalid request payloads are automatically rejected.

30. Explain async and await in Python.

Answer:

async and await allow cooperative concurrency.

They're particularly useful for I/O operations such as:

- HTTP requests
- Database access
- LLM calls
- File/network operations

Example:

```
async def fetch_model_response():
    response = await client.call_model()
    return response
```

They do not automatically make CPU-intensive Python operations faster.

CPU-heavy workloads may need processes, worker queues, or specialized compute.

31. What is Pydantic?

Answer:

Pydantic validates and serializes Python data based on type annotations.

```
from pydantic import BaseModel
```

```
class AgentRequest(BaseModel):
    query: str
    user_id: int
```

If an invalid `user_id` is provided, Pydantic catches the problem before application logic executes.

It's valuable in AI applications because LLM-generated or external data should not be blindly trusted.

AI / LangGraph / Agentic Systems

32. What is the difference between an LLM application and an AI agent?

Answer:

A basic LLM application may simply perform:

input → prompt → model → response

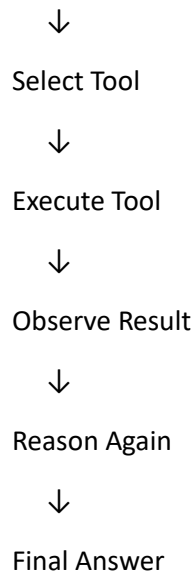
An agent includes a control loop where the model can decide what action to take.

For example:

User Request

↓

Reason



The important difference is that an agent can operate over state and make multiple decisions instead of performing one inference call.

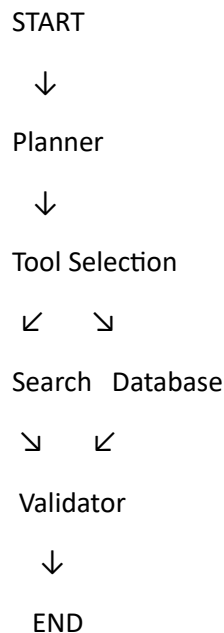
33. What is LangGraph?

Answer:

LangGraph is a framework for building stateful, graph-based LLM workflows.

Instead of representing an application as a simple chain, we model it as nodes and edges.

Example:



Each node can inspect and update shared state.

LangGraph is particularly useful when you need branching, loops, retries, persistence, human approval, or long-running agent workflows.

34. What is state in LangGraph?

Answer:

State represents the data flowing through the graph.

For example:

```
class AgentState(TypedDict):  
    messages: list  
    user_id: str  
    selected_tool: str  
    tool_result: dict  
    retry_count: int
```

Individual nodes read from and update this shared state.

Good state design matters because putting excessive or poorly structured information in state can make the workflow hard to understand and expensive to persist.

35. What is checkpointing in LangGraph?

Answer:

Checkpointing persists graph state so that execution can resume later.

Imagine a workflow:

Analyze

↓

Request human approval

↓

Execute action

The process might stop for several hours while waiting for approval.

Without checkpointing, we would need to rebuild the complete execution context.

With checkpointing, the application can restore the previous graph state and continue.

Checkpointing also helps with failure recovery and conversational persistence.

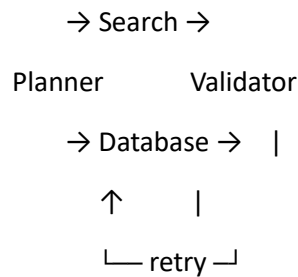
36. When would you use a graph rather than a chain?

Answer:

A chain works well for predictable sequential workflows:

$A \rightarrow B \rightarrow C$

A graph is more appropriate when execution can branch or loop:



Agent systems naturally have branching and retry paths, which is why graph-based orchestration is useful.

37. How do you handle malformed LLM output?

Answer:

I never assume model output is valid simply because I requested a particular structure.

I use multiple layers:

1. Request structured output
2. Validate it against a schema
3. Retry when appropriate
4. Apply sensible fallback behavior
5. Log validation errors
6. Fail safely for high-risk actions

For example, with Pydantic:

```
class ToolDecision(BaseModel): tool: str arguments: dict
```

If parsing fails, I don't blindly pass the output to another system.

38. How should tool calling be implemented safely?

Answer:

The LLM should request a tool operation, but application code should decide whether that operation is permitted.

For example, the model might produce:

```
{  
  "tool": "delete_user",  
  "user_id": 123
```

}

Before executing it, the application should validate:

- Tool exists
- Arguments are valid
- Caller has permission
- Resource belongs to the correct tenant
- Operation doesn't violate business rules

LLM authorization is never a substitute for application authorization.

39. How would you prevent an agent from entering an infinite loop?

Answer:

I'd enforce deterministic safeguards outside the model.

For example:

max iterations = 10

max tool calls = 20

timeout = 60 seconds

I'd also maintain state such as:

retry_count

previous_actions

tool_errors

If the agent repeats the same action without making progress, the workflow should terminate or route to fallback/human intervention.

40. What is RAG?

Answer:

RAG stands for Retrieval-Augmented Generation.

The basic process is:

Question

↓

Embedding / Search

↓

Relevant Documents

↓

Prompt + Documents

↓

LLM

↓

Answer

It helps ground responses in external knowledge without retraining the model.

In production I would consider retrieval quality, chunking, metadata filtering, permissions, reranking, citation quality, and evaluation rather than simply plugging in a vector database.

AWS Bedrock / Claude

41. What is AWS Bedrock?

Answer:

AWS Bedrock is AWS's managed service for accessing foundation models from providers such as Anthropic and others through AWS infrastructure.

In this JD, Claude models are accessed through Bedrock.

A typical flow is:

Application

↓

AWS SDK

↓

Amazon Bedrock

↓

Claude model

Advantages can include centralized AWS IAM controls, model access management, logging integration, and keeping AI infrastructure within an AWS environment.

42. If you have OpenAI experience but not Bedrock experience, how transferable is it?

Answer:

Very transferable at the application layer.

The major concepts remain:

- Prompt design
- Token management

- Structured output
- Tool calling
- Streaming
- Retries
- Rate limits
- Context windows
- Model errors

The biggest differences are usually authentication, SDK APIs, model-specific payloads, deployment policies, IAM, and provider-specific capabilities.

So I would expect to ramp up quickly if I already understand production LLM application architecture.

Testing

43. How would you test role-based permissions using Playwright?

Answer:

I'd create separate authenticated contexts for each role.

For example:

```
test("viewer cannot delete user", async ({ browser }) => {
  const context = await browser.newContext({
    storageState: "viewer.json"
  });

  const page = await context.newPage();

  await page.goto("/users");

  await expect(
    page.getByRole("button", { name: "Delete" })
  ).not.toBeVisible();
});
```

But I would also test the backend API directly because hiding the UI is not sufficient:

```
const response = await request.delete("/api/users/123");
```

```
expect(response.status()).toBe(403);
```

This validates both UX and actual security.

44. What is the difference between unit, integration, API, and E2E tests?

Answer:

A **unit test** isolates a small unit such as a function.

An **integration test** verifies multiple components working together.

An **API test** calls endpoints directly and verifies contracts, permissions, and business behavior.

An **E2E test** simulates real user behavior through the application.

I prefer a layered test strategy rather than trying to test everything through slow E2E tests.

Git / GitHub Actions

45. What is the difference between merge and rebase?

Answer:

Merge combines histories and usually creates a merge commit.

A---B---C

\

D---E

After merge:

A---B---C---M

\ /

D---E

Rebase moves commits onto another base:

A---B---C---D'---E'

Rebase creates a cleaner linear history but rewrites commit hashes.

I avoid rebasing commits that other developers already depend on unless the team's workflow explicitly expects it.

46. How do you resolve a Git conflict?

Answer:

First I understand what each side intended instead of mechanically selecting "ours" or "theirs."

Typical process:

`git fetch`

`git rebase origin/main`

Git marks conflicting files.

I resolve each file manually, then:

`git add file.js`

`git rebase --continue`

Afterwards I run relevant tests because a syntactically resolved conflict can still create logical defects.

47. A GitHub Actions pipeline suddenly fails. What do you do?

Answer:

I first identify the failing job and exact failing step.

Then I determine whether it's:

- Test failure
- Build failure
- Dependency issue
- Environment variable issue
- Secret/permission problem
- Cache problem
- Runner/environment change
- Deployment failure

For example, if:

`npm test`

fails, I reproduce that command locally using the same runtime versions where possible.

I avoid immediately changing the workflow until I understand whether the pipeline or the application is actually broken.

Docker

48. What Docker Compose commands do you commonly use?

Answer:

Start services:

`docker compose up`

Run in background:

`docker compose up -d`

View logs:

`docker compose logs`

Follow logs:

`docker compose logs -f api`

List services:

`docker compose ps`

Enter a running container:

`docker compose exec api sh`

Restart:

`docker compose restart api`

Stop everything:

`docker compose down`

For this role, the important skill is being able to diagnose why a local multi-service environment is not working, rather than merely memorizing Docker syntax.

AI Coding Agents / Claude Code

49. How do you use Claude Code or another coding agent responsibly?

Answer:

I treat an AI coding agent like a fast junior collaborator rather than an authority.

I give it bounded tasks, for example:

Inspect these three files, identify why this endpoint returns 403 for managers, propose the smallest fix, and do not modify unrelated code.

rather than:

Fix the application.

I then review:

- Correctness
- Security
- Edge cases
- Existing project conventions
- Tests
- Unnecessary changes

- Hallucinated APIs
- Performance implications

Most importantly, I don't submit code that I cannot explain myself.

50. Tell me about a time an AI coding assistant was confidently wrong. How did you catch it?

This question is **extremely likely** because the JD explicitly calls it out.

A good answer would be:

Answer:

I was using an AI coding assistant to modify an API integration. It generated code that looked correct and used a plausible SDK method, but during review I noticed that the method didn't exist in the version of the SDK used by the project.

Instead of accepting it, I checked the installed package version and existing implementation, confirmed the actual API, and corrected the generated code.

I then added a test covering that path.

That experience reinforced how I use coding agents: they are excellent for accelerating investigation, boilerplate, and implementation, but generated code still has to pass the same review standard as human-written code. I verify unfamiliar APIs, review diffs carefully, and make sure I can defend every line before merging it.

10 Questions I Would Prepare Most Deeply

If your interview is soon, prioritize these **10 first** because they map almost word-for-word to the JD:

1. **Explain prototype-based inheritance in ES5 JavaScript.**
2. **Explain Shadow DOM and how styles cross the boundary.**
3. **What is Lit and how does its reactive rendering work?**
4. **Authentication vs authorization, and how would you implement RBAC?**
5. **What is LangGraph and why use a graph instead of a chain?**
6. **Explain LangGraph state and checkpointing.**
7. **What makes something an agent rather than just an LLM API call?**
8. **How do you safely validate tool calls and structured LLM output?**
9. **How would you test different user roles using Playwright?**
10. **Describe a time an AI coding agent generated incorrect code and how you caught it.**

One important interview strategy

For this particular JD, **do not present yourself as only an AI engineer**. The role is clearly looking for someone who can survive a mature production codebase.

Your answers should repeatedly demonstrate this combination:

Legacy ES5 JavaScript + modern TypeScript/UI + REST/database/security + Python/FastAPI + genuine LangGraph agent engineering + testing/delivery discipline.

That combination is probably what will separate a strong candidate from someone who has only built a few LLM demos.